EECE 2323 Digital Logic Design Lab

Name: Arefin Mahmood

Lab Report 2: Arithmetic and Logic Unit

Date: 05/29/2026

Background & Purpose

In this lab, I designed and tested a 16-bit Arithmetic and Logic Unit, or ALU, using SystemVerilog and Vivado. The ALU uses two signed 16-bit inputs, a and b, and a 4-bit selector, s, to decide which operation to run. The result is sent to f. The design also includes ovf for signed overflow and take_branch for branch operations.

This lab builds on the 16-bit adder from the previous lab by adding more processor functions. Instead of only doing addition, the ALU can now do arithmetic, logic, shift, and branch condition operations. This matters because an ALU is one of the main parts of a processor datapath, and these same types of operations are used later when the datapath becomes part of a larger processor.

Design Analysis

In Lab 1, the design was only a 16-bit signed adder. For this lab, I built on that and made a 16-bit ALU that can do more than one operation. The ALU supports ADD, INV, AND, OR, SRA, SLL, BEQZ, BNEZ, and XOR. The selector input s decides which operation is used. I used an always_comb block with a case statement because the ALU is a combinational circuit, so the outputs should change based on the current inputs only.

The main file I wrote for the design was alu.sv. At the start of the always_comb block, I gave f, ovf, and take_branch default values. This way, the outputs are always defined before the case statement runs. It also makes the unused selector values safer. Since s is 4 bits, it can represent 16 possible values, but only 9 operations are used in this lab. For any unused selector value, the default case sets f = 16'h0000, ovf = 0, and take_branch = 0. For the ADD operation, I also included signed overflow detection. In my design, ovf turns on when a and b have the same sign, but the result f has a different sign.

Results and Analysis

I first tested the ALU with the testbench shown in Appendix B.2. The simulation waveform is shown in Appendix A, Figure 1. In the testbench, each input case ran for 100 ns before moving to the next one. I used the simulation to check normal addition, positive overflow, negative overflow, inversion, AND, OR, arithmetic right shift, logical left shift, BEQZ, BNEZ, XOR, and the default unused selector case. From the waveform, the f output, ovf flag, and take_branch flag changed the way I expected for each selector value.

After the simulation worked, I programmed the PYNQ board and tested the ALU in hardware using VIO. I used VIO because the ALU needs 36 input bits total: 16 bits for a, 16 bits for b, and 4 bits for s. The board does not have enough switches for all of those inputs, so VIO let me type the values directly and check the output signals. The hardware tests matched my expected results. For example, 0003 + 0004 gave 0007 with ovf = 0. For overflow, 7FFF + 0001 gave 8000 and ovf changed to 1. The SRA test also worked because FFF0 shifted right by 2 gave FFFC, which means the sign bit was kept. The branch operations worked too. BEQZ only set take_branch to 1 when a was 0000, and BNEZ set take_branch to 1 when a was not 0000.

Conclusion & Recommendations

This lab helped me see how the 16-bit adder from Lab 1 fits into a bigger ALU design. Instead of only adding two numbers, the ALU now uses the selector input s to choose between arithmetic, logic, shift, and branch operations. I also learned that overflow should only be checked during signed addition. For logic and shift operations, ovf should stay 0 because those operations work bit by bit and do not create signed overflow.

The simulation and VIO hardware tests both showed the ALU working for the required operations. The main cases I had to pay attention to were signed overflow, arithmetic right shift with a negative number, and the branch outputs for BEQZ and BNEZ. For future labs, I would test each operation in simulation before moving to the board, especially signed values and branch cases, because small mistakes there can change the output completely.

Appendix A: Prelab and Test Data/Simulation

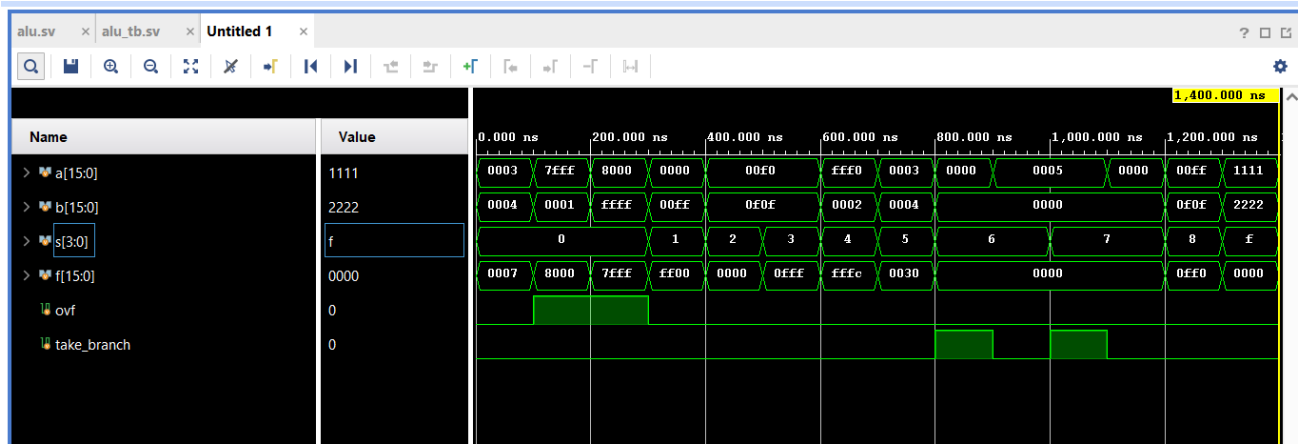


Figure A1: Behavioral simulation waveform for alu_tb.sv

Test	a	b	s	Operation	Expected f	Expected ovf	Expected take_branch
1	16'h0003	16'h0004	4'h0	ADD normal	16'h0007	0	0
2	16'h7FFF	16'h0001	4'h0	ADD positive overflow	16'h8000	1	0
3	16'h8000	16'hFFFF	4'h0	ADD negative overflow	16'h7FFF	1	0
4	16'h0000	16'h00FF	4'h1	INV	16'hFF00	0	0
5	16'h00F0	16'h0F0F	4'h2	AND	16'h0000	0	0
6	16'h00F0	16'h0F0F	4'h3	OR	16'h0FFF	0	0
7	16'hFFF0	16'h0002	4'h4	SRA	16'hFFFC	0	0
8	16'h0003	16'h0004	4'h5	SLL	16'h0030	0	0
9	16'h0000	16'h0000	4'h6	BEQZ true	16'h0000	0	1
10	16'h0005	16'h0000	4'h6	BEQZ false	16'h0000	0	0
11	16'h0005	16'h0000	4'h7	BNEZ true	16'h0000	0	1
12	16'h0000	16'h0000	4'h7	BNEZ false	16'h0000	0	0
13	16'h00FF	16'h0F0F	4'h8	XOR	16'h0FF0	0	0
14	16'h1111	16'h2222	4'hF	Default case	16'h0000	0	0

Table A1: Main ALU Hardware Test Values

Appendix B: Design Program Files

B.1 ALU Module

```
`timescale 1ns / 1ps
module alu(
    input  logic signed [15:0] a,
    input  logic signed [15:0] b,
    input  logic [3:0] s,
    output logic signed [15:0] f,
    output logic ovf,
    output logic take_branch
);
```

```
always_comb begin
        // Default values
        f = 16'h0000;
        ovf = 1'b0;
        take_branch = 1'b0;
        case (s)
            // ADD: f = a + b
            4'b0000: begin
                f = a + b;
                // Overflow happens when a and b have the same sign,
                // but the result has a different sign.
                if ((a[15] == b[15]) && (f[15] != a[15]))
                    ovf = 1'b1;
                else
                    ovf = 1'b0;
                take_branch = 1'b0;
            end
            // INV: f = bitwise inversion of b
            4'b0001: begin
                f = ~b;
                ovf = 1'b0;
                take_branch = 1'b0;
            end
            // AND: f = a AND b
            4'b0010: begin
                f = a & b;
                ovf = 1'b0;
                take_branch = 1'b0;
            end
            // OR: f = a OR b
            4'b0011: begin
                f = a | b;
                ovf = 1'b0;
                take_branch = 1'b0;
            end
            // SRA: arithmetic right shift
            4'b0100: begin
```

```
    f = a >>> b;
    ovf = 1'b0;
    take_branch = 1'b0;
end
// SLL: logical left shift
4'b0101: begin
    f = a << b;
    ovf = 1'b0;
    take_branch = 1'b0;
end
// BEQZ: branch if a equals zero
4'b0110: begin
    f = 16'h0000;
    ovf = 1'b0;

    if (a == 16'h0000)
        take_branch = 1'b1;
    else
        take_branch = 1'b0;
end

// BNEZ: branch if a does not equal zero
4'b0111: begin
    f = 16'h0000;
    ovf = 1'b0;

    if (a != 16'h0000)
        take_branch = 1'b1;
    else
        take_branch = 1'b0;
end

// XOR: f = a XOR b
4'b1000: begin
    f = a ^ b;
    ovf = 1'b0;
    take_branch = 1'b0;
```

```
            end

            // Any unused selector value
            default: begin
                f = 16'h0000;
                ovf = 1'b0;
                take_branch = 1'b0;
            end

        endcase
    end
endmodule
```

B.2 ALU Testbench

```
`timescale 1ns / 1ps
module alu_tb;
    logic signed [15:0] a;
    logic signed [15:0] b;
    logic [3:0] s;
    logic signed [15:0] f;
    logic ovf;
    logic take_branch;

    alu dut (
        .a(a),
        .b(b),
        .s(s),
        .f(f),
        .ovf(ovf),
        .take_branch(take_branch)
    );
    initial begin
        // ADD normal
        a = 16'h0003; b = 16'h0004; s = 4'b0000;
        // ADD positive overflow
        #100 a = 16'h7FFF; b = 16'h0001; s = 4'b0000;
        // ADD negative overflow
        #100 a = 16'h8000; b = 16'hFFFF; s = 4'b0000;
```

```
        // INV
        #100 a = 16'h0000; b = 16'h00FF; s = 4'b0001;
        // AND
        #100 a = 16'h00F0; b = 16'h0F0F; s = 4'b0010;
        // OR
        #100 a = 16'h00F0; b = 16'h0F0F; s = 4'b0011;
        // SRA
        #100 a = 16'hFFF0; b = 16'h0002; s = 4'b0100;
        // SLL
        #100 a = 16'h0003; b = 16'h0004; s = 4'b0101;
        // BEQZ true
        #100 a = 16'h0000; b = 16'h0000; s = 4'b0110;
        // BEQZ false
        #100 a = 16'h0005; b = 16'h0000; s = 4'b0110;
        // BNEZ true
        #100 a = 16'h0005; b = 16'h0000; s = 4'b0111;
        // BNEZ false
        #100 a = 16'h0000; b = 16'h0000; s = 4'b0111;
        // XOR
        #100 a = 16'h00FF; b = 16'h0F0F; s = 4'b1000;
        // Default unused selector
        #100 a = 16'h1111; b = 16'h2222; s = 4'b1111;
        #100 $stop;
    end
endmodule
```

Appendix C: Output Screen Captures

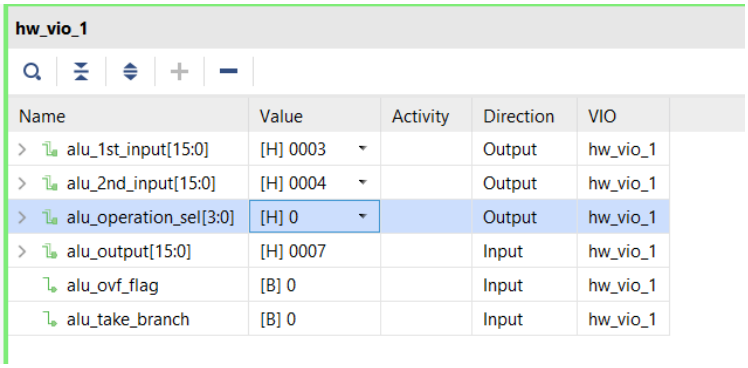


Figure C1: VIO hardware result for ADD normal.

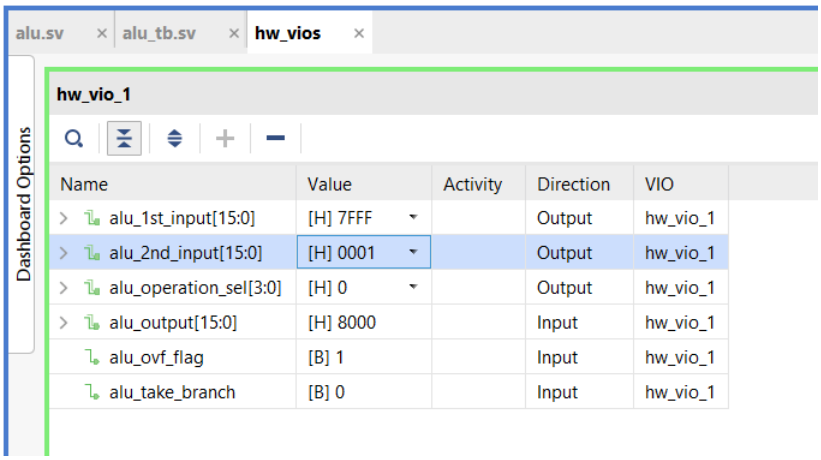


Figure C2: VIO hardware result for positive overflow.

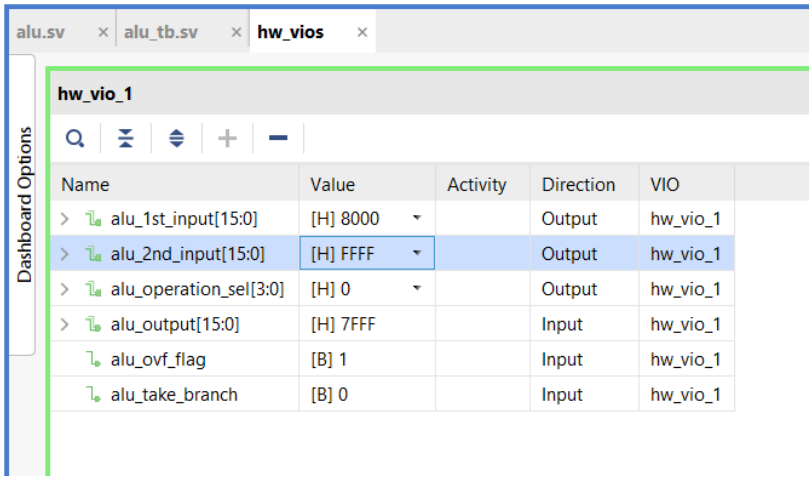


Figure C3: VIO hardware result for negative overflow.

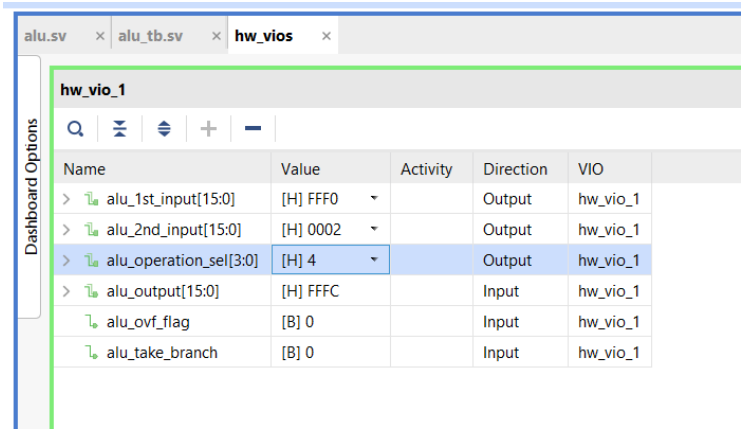


Figure C4: VIO hardware result for arithmetic right shift.

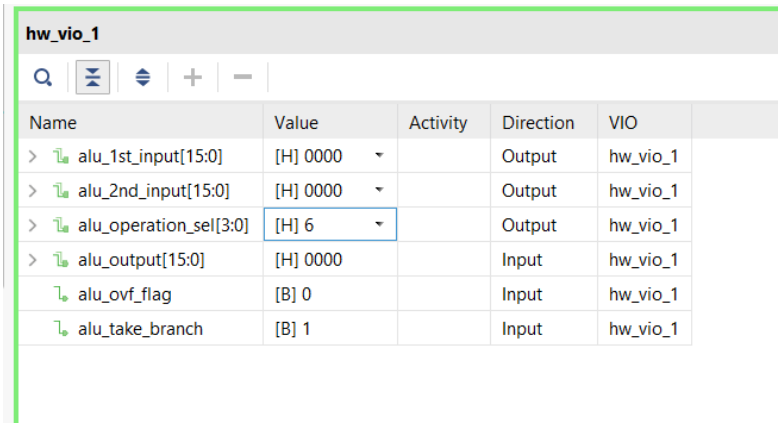


Figure C5: VIO hardware result for BEQZ true.

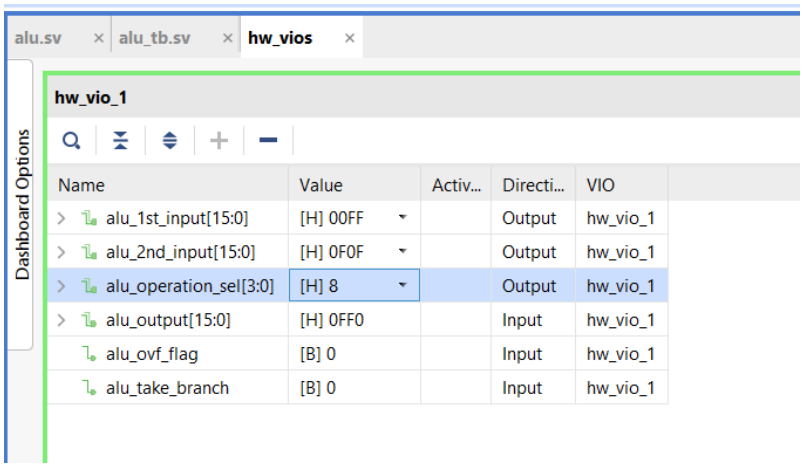


Figure C6: VIO hardware result for XOR.

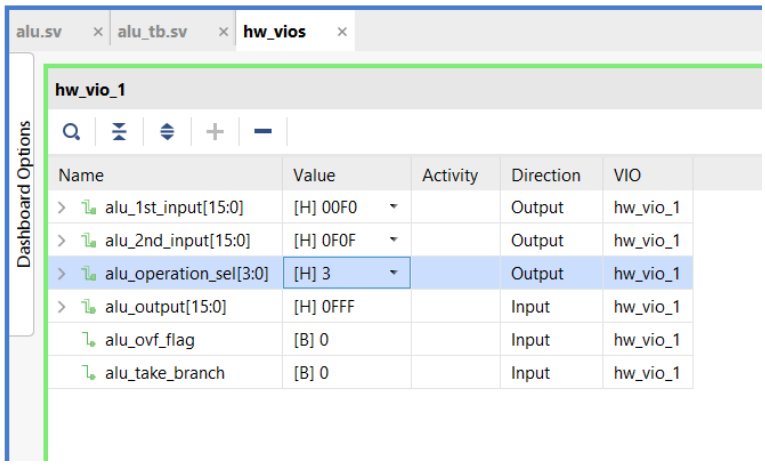


Figure C7: VIO hardware result for OR operation

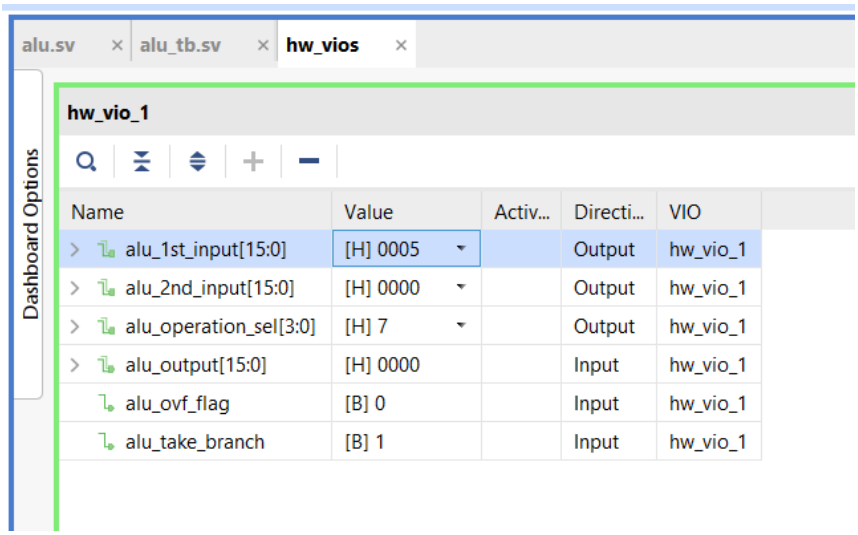


Figure C8: VIO hardware result for BNEZ true